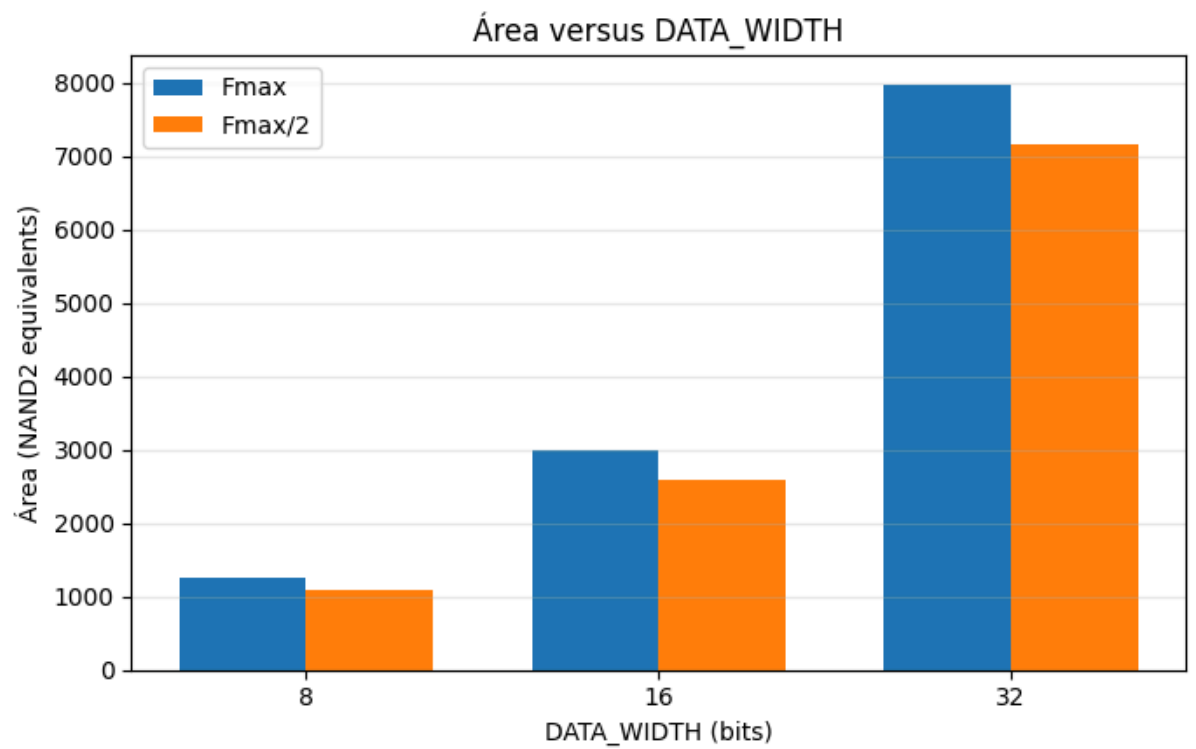
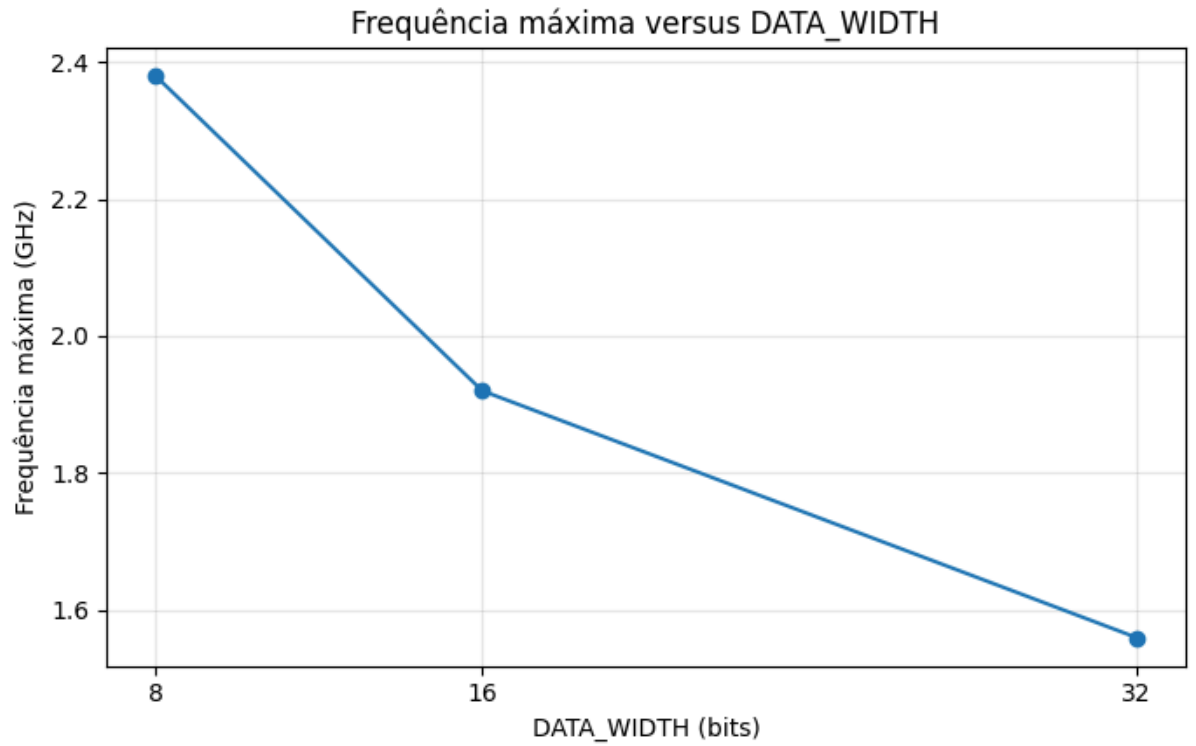


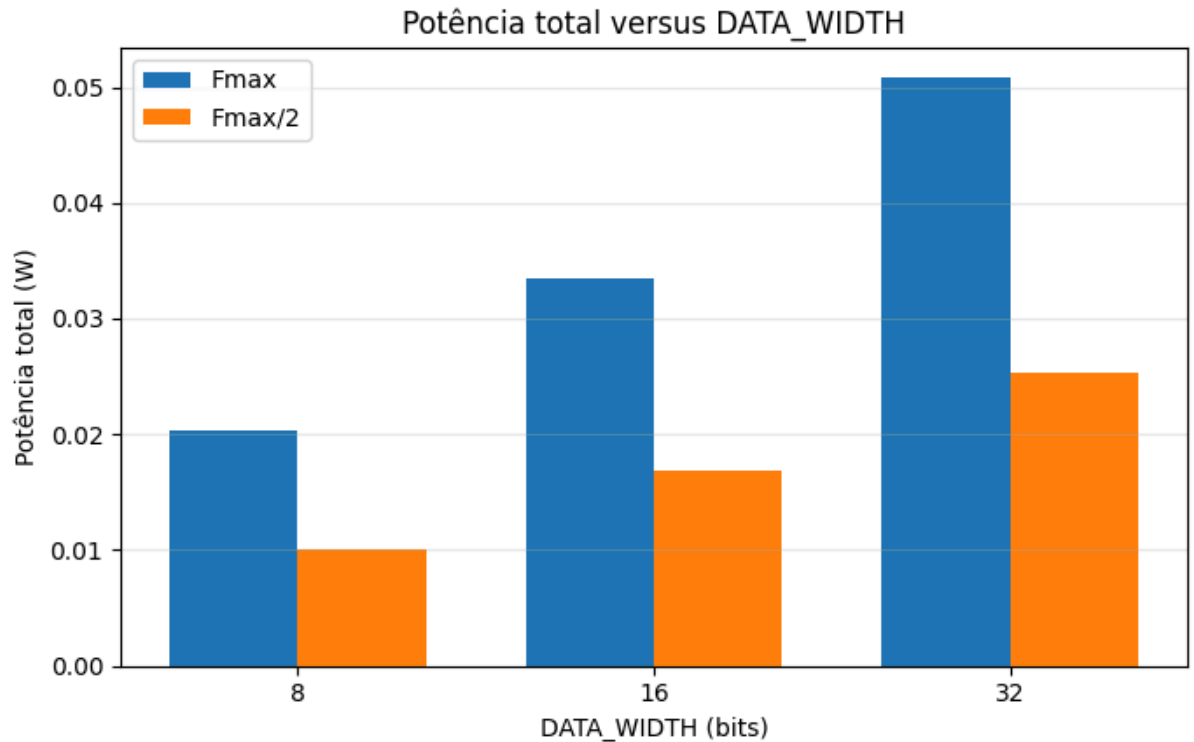
Comparação dos resultados:

Para cada experimento foi executado o script tcl com uma constraint específica e um circuito específico variando o data_width e clock para obter um slack 0. A tabela a seguir apresenta os resultados: Para comparação com a NAND2 foi usado a NAND2: NAND2X1, que possui area : 1.026, na biblioteca fast_vdd1v0_basicCells.lib

Tabela – Resultados de síntese

Parâmetro	E1	E2	E3	E4	E5	E6
Bits	8	8	16	16	32	32
Frequência-alvo	Fmax	Fmax/2	Fmax	Fmax/2	Fmax	Fmax/2
Frequência obtida	2,38 GHz	1,19 GHz	1,92 GHz	0,96 GHz	1,56 GHz	0,78 GHz
WNS	0	0	0	0	0	0
Área total	1285,304	1106,028	3076,290	2662,812	8190,900	7340,688
Área equivalente em NAND	1252,733	1078,000	2998,333	2595,333	7983,333	7154,667
Potência total	$2,03973 \times 10^{-2} \text{ W}$	$1,01090 \times 10^{-2} \text{ W}$	$3,34948 \times 10^{-2} \text{ W}$	$1,67993 \times 10^{-2} \text{ W}$	$5,08814 \times 10^{-2} \text{ W}$	$2,52524 \times 10^{-2} \text{ W}$
Número de células	502	343	1273	952	3295	2594





1) Como a largura dos operandos afeta o caminho crítico e a frequência máxima?

O aumento da largura dos operandos aumenta a complexidade da operação de multiplicação e acumulação. Para operandos maiores, são gerados mais produtos parciais, mais níveis de redução na árvore de soma e uma cadeia de propagação de carry mais longa. Como consequência, o atraso do caminho crítico aumenta e a frequência máxima diminui.

2) A síntese em metade da frequência altera a seleção ou o tamanho das células? Como isso afeta a área?

A redução da frequência-alvo altera tanto a seleção quanto, em alguns casos, a quantidade de células. Nos experimentos em Fmax, a síntese emprega células com maior capacidade de acionamento, como versões **X2**, **X4** e **X6**, para satisfazer as restrições mais severas de temporização. Em Fmax/2, a maior folga temporal permite o uso de células menores, frequentemente identificadas pelo sufixo **XL**, além de possibilitar uma implementação com menos células. Como consequência, a área em equivalentes NAND2 é reduzida em aproximadamente 14,0%, 13,4% e 10,4% para as configurações de 8, 16 e 32 bits, respectivamente.

3) Qual componente de potência domina em cada configuração?

Switching é a componente que domina em todas as configurações, sendo responsável por em média 90% do consumo do circuito.

4) A potência diminui aproximadamente de forma linear com a frequência? Justifique eventuais diferenças.

Quase linear, devido a potência Switching cair pela metade e ela representar quase 90% da potência do custo da potência. Mas os outros 10% reduzem devido à mudança no circuito por usar células lógicas menores e com menos consumo. Por isso não é exatamente linear, embora se aproxime devido aos 90%.

5) Quais operações e células aparecem no caminho crítico?

O caminho crítico está concentrado na lógica combinacional da operação MAC, especialmente na redução dos produtos parciais da multiplicação e na soma final. Os nomes WALLACE_CSA_DUMMY_OP e csa_tree_ADD_TC_OP indicam a presença de árvores Wallace e carry-save adders. Entre as células mais recorrentes estão os somadores completos ADDFX1, ADDFXL e ADDFHXL, além de portas XOR/XNOR, NAND, NOR e células compostas AOI/OAI. Em vários caminhos, os arcos CI->CO dos full adders mostram que a propagação de carry contribui significativamente para o atraso crítico

6) Os resultados observados são coerentes com a arquitetura da unidade MAC?

Sim.

Os gráficos foram gerados a partir do código python:

```
import matplotlib.pyplot as plt
import numpy as np

# =====
# Dados dos experimentos
# =====

data_width = np.array([8, 16, 32])

# Frequência máxima obtida para cada DATA_WIDTH, em GHz
frequencia_fmax = np.array([2.38, 1.92, 1.56])

# Área em equivalentes NAND2
area_fmax = np.array([
    1252.733,
    2998.333,
    7983.333
])

area_fmax_2 = np.array([
    1078.000,
    2595.333,
```

```

    7154.667
])

# Potência total, em watts
potencia_fmax = np.array([
    2.03973e-2,
    3.34948e-2,
    5.08814e-2
])

potencia_fmax_2 = np.array([
    1.01090e-2,
    1.67993e-2,
    2.52524e-2
])

# =====
# Gráfico 1: frequência máxima versus DATA_WIDTH
# =====

plt.figure(figsize=(7, 4.5))

plt.plot(
    data_width,
    frequencia_fmax,
    marker="o",
    linewidth=2
)

plt.xlabel("DATA_WIDTH (bits)")
plt.ylabel("Frequência máxima (GHz)")
plt.title("Frequência máxima versus DATA_WIDTH")

plt.xticks(data_width)
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(
    "frequencia_maxima_vs_data_width.png",
    dpi=300,
    bbox_inches="tight"
)
plt.show()

# =====
# Gráfico 2: área em NAND2 equivalentes versus DATA_WIDTH

```

```
# =====

posicoes = np.arange(len(data_width))
largura_barra = 0.36

plt.figure(figsize=(7, 4.5))

plt.bar(
    posicoes - largura_barra / 2,
    area_fmax,
    width=largura_barra,
    label="Fmax"
)

plt.bar(
    posicoes + largura_barra / 2,
    area_fmax_2,
    width=largura_barra,
    label="Fmax/2"
)

plt.xlabel("DATA_WIDTH (bits)")
plt.ylabel("Área (NAND2 equivalents)")
plt.title("Área versus DATA_WIDTH")

plt.xticks(posicoes, data_width)
plt.legend()
plt.grid(True, axis="y", alpha=0.3)

plt.tight_layout()
plt.savefig(
    "area_nand2_vs_data_width.png",
    dpi=300,
    bbox_inches="tight"
)
plt.show()
```

```
# =====
# Gráfico 3: potência total versus DATA_WIDTH
# =====
```

```
plt.figure(figsize=(7, 4.5))

plt.bar(
    posicoes - largura_barra / 2,
    potencia_fmax,
    width=largura_barra,
```

```

        label="Fmax"
    )

    plt.bar(
        posicoes + largura_barra / 2,
        potencia_fmax_2,
        width=largura_barra,
        label="Fmax/2"
    )

    plt.xlabel("DATA_WIDTH (bits)")
    plt.ylabel("Potência total (W)")
    plt.title("Potência total versus DATA_WIDTH")

    plt.xticks(posicoes, data_width)
    plt.legend()
    plt.grid(True, axis="y", alpha=0.3)

    plt.tight_layout()
    plt.savefig(
        "potencia_total_vs_data_width.png",
        dpi=300,
        bbox_inches="tight"
    )
    plt.show()

```